Диапазон таймаута выборов в Raft при потере пакетов: реализация, детерминированный симулятор и проверка оценок Ongaro

1. Вопрос

В Raft лидер выбирается по таймауту, который каждый сервер берёт случайно из диапазона $[T, T + W]$. Диссертация Ongaro [2] даёт формулу для раскола голосов (split vote) и оценивает число сроков до лидера геометрическим распределением, но только для сети без потерь. Среди вопросов, оставленных на будущее, в конце главы 9 стоит «как работают выборы при сильной потере пакетов».

Я спросил так: как ширина диапазона W влияет на время до лидера и на долю неудачных сроков, когда каждое сообщение теряется с вероятностью p от 0 до 0,5, и при каких условиях оценки из [2] перестают работать. Вопрос закрыт, если выполнены три вещи. Моя реализация вместе с симулятором воспроизводит формулу Ongaro при $p = 0$ в пределах статистической ошибки. При $p > 0$ измерены время до лидера и доля сроков, провалившихся из-за раскола и из-за потери запросов. Для каждого расхождения с [2] названо условие, при котором оно возникает.

Базовая постановка такая: пять серверов, односторонняя задержка равномерно от 5 до 10 мс, потери независимые, $T = 150$ мс, период heartbeat 50 мс, W от 2 до 600 мс. Размеры кластера 3, 7 и 9 и другие масштабы задержки я проверил отдельно.

2. Что уже известно

Алгоритм описан в статье Ongaro и Ousterhout [1]. Я читал расширенную версию. Раздел 5.2 рекомендует таймауты вроде 150–300 мс, а раздел 9.3 измеряет простой после падения лидера в кластере из пяти серверов с временем рассылки около 15 мс. Без случайности выборы «стабильно занимали больше 10 секунд» из-за раскола голосов. Добавка 5 мс случайности давала медиану простоя 287 мс, а при 50 мс худший случай из 1000 испытаний составил 513 мс. С таймаутом 12–24 мс лидер выбирался в среднем за 35 мс. Потерь пакетов в этих измерениях нет.

Диссертация [2] разбирает выборы подробнее. При фиксированной задержке l (доля диапазона) раскол при s рабочих серверах из n происходит, если не меньше $c = s - \lfloor n/2 \rfloor + 1$ серверов сработают в пределах l друг от друга. Вероятность этого равна

$$P(D_{c,s} \le l) = \Sigma_{k=c-1}^{s}\, \mathrm{C}(s,k)\, l^{k}(1-l)^{s-k}. \tag{1}$$

Для переменной задержки диссертация даёт симуляцию из трёх шагов. Число сроков до лидера Ongaro считает геометрическим, потому что серверы сбрасывают таймеры при голосовании и следующий срок «по сути без памяти». Рекомендуемый диапазон составляет 10–20 односторонних задержек. Расширение PreVote описано в разделе 9.6 [2]. Таймауты кандидатов отдельно от таймаутов последователей предлагал Howard [4], его работу я знаю только по ссылке в [2]. Невозможность детерминированного консенсуса в асинхронной модели доказана в [3].

Лабораторные курса MIT 6.5840 [5] задают типовой набор тестов Raft (выборы, репликация, перезапуски, Figure 8, ненадёжная сеть). Страницу курса из облачной машины открыть не удалось, сетевая политика закрывает домен. Поэтому названия тестов и общую схему сценариев я воспроизвожу по памяти и точного соответствия лабораторным не заявляю.

3. Что я сделал

3.1. Реализация

Узел Raft написан на C++20 и занимает около 450 строк вместе с заголовком. Он получает сообщения и пробуждения по таймеру, а наружу отдаёт сообщения, применённые записи и смены роли, поэтому о времени и сети ничего не знает. Реализованы выборы со случайными таймаутами, репликация журнала с быстрым откатом (ответ несёт срок и первый индекс конфликтующей записи, как описано в разделе 5.3 [1]), хранение срока, голоса и журнала через перезапуск. Снимков, смены состава и клиентских сессий нет. Новый лидер не пишет пустую запись, и поэтому записи старых сроков фиксируются вместе с первой новой командой. Лидер не проверяет связь с большинством и уходит только при виде большего срока.

К базовому алгоритму я добавил два переключателя, которых в [1] нет. Первый называется PreVote и сделан по описанию из раздела 9.6 [2]: кандидат повышает срок только после того, как большинство обещало голос, а обещают его серверы, не слышавшие лидера дольше минимального таймаута. Второй переключатель повторяет RequestVote тем, кто ещё не ответил, каждые 40 мс внутри того же срока. Базовый Raft шлёт каждый запрос голоса один раз. Повтор придуман мной, в литературе я его не искал.

Ещё четыре переключателя ломают алгоритм намеренно, для проверки самих тестов: подсчёт реплик для записей старых сроков (ошибка Figure 8), голос без проверки свежести журнала, забывание голоса при перезапуске и усечение журнала при любом AppendEntries.

3.2. Симулятор

Время виртуальное, в микросекундах, события лежат в очереди с порядковым номером для разрешения ничьих. Сеть задаёт задержку из равномерного диапазона, независимую потерю каждого сообщения, дубли, редкую длинную задержку для переупорядочения, разрыв связи между парами и изоляцию узла. Узел можно уронить и вернуть, и тогда он теряет всё, кроме срока, голоса и журнала. Диск моделируется без задержки: запись мгновенна. Генератор случайных чисел (xoshiro256** с разгоном через splitmix64) написан мной, потому что распределения стандартной библиотеки C++ не определены одинаково на разных платформах, а мне нужны побайтно одинаковые результаты. Статистических тестов качества генератора я не запускал.

Рядом с симулятором работает проверяющий модуль. Он не смотрит внутрь узлов и опирается только на события наружу и на стабильные журналы. Он проверяет, что в одном сроке не больше одного лидера, что каждый новый лидер уже содержит все применённые записи, что на одном индексе все узлы применили одну команду, что журналы с одинаковым сроком на индексе совпадают до него, что сроки на узле не убывают и что применение идёт подряд.

3.3. Корректность

Сценарных тестов 20. Это начальные выборы, перевыборы, много выборов, согласие на команде, отказ последователя, отказ без большинства, одновременные запуски, возврат изолированного лидера, быстрый откат журнала, три сценария перезапусков, три варианта Figure 8, ненадёжная сеть, поток отказов, сохранение голоса, помеха от возвращающегося последователя и детерминированность. Каждый тест идёт на 20 начальных значениях генератора в четырёх конфигурациях (PreVote и повтор запросов включены или выключены). Это 1600 прогонов и 51 370 проверок, и все проходят. Сценарий быстрого отката с 50 конфликтующими записями занимает 9,5–11 с виртуального времени и 3,3–4,3 тысячи сообщений.

Прохождение тестов ничего не доказывает, пока не видно, что они что-то ловят. Поэтому я прогнал те же тесты и случайный фаззер на четырёх сломанных вариантах (таблица 1). Фаззер строит случайные сценарии из 3, 5 или 7 узлов с потерями до 30%, дублями, переупорядочением, разрывами

связи, падениями и возвратами, по 40 с виртуального времени. На корректном коде 40 000 сценариев (20 000 без PreVote и 20 000 с ним) дали ноль нарушений, это примерно 18 суток виртуального времени.

Таблица 1. Сломанные варианты узла. Тесты: сколько из 20 сценариев падают хотя бы в одной из четырёх конфигураций. Фаззер: сколько из 5000 случайных сценариев нарушают инвариант.

вариант	тесты, упало	какие тесты ловят	фаззер, нарушений из 5000
считать реплики старых сроков	1	Figure8Paper	5 (Leader Completeness)
голос без проверки журнала	8	Backup, Churn, FailAgree, Figure8, Figure8Unreliable, Persist2, Persist3, Rejoin	4297
забывать голос при перезапуске	1	VotePersistence	1 (Election Safety)
усекать журнал при любом AppendEntries	14	BasicAgree, FailAgree, FailNoAgree, ConcurrentStarts, Rejoin, Backup, Persist1–3, три Figure 8, Churn, UnreliableAgree	1387 (из них 318 лавин сообщений)

Первая версия набора не ловила два самых тонких варианта: ошибку с репликами старых сроков и забытый голос не нашёл ни один из 18 тогдашних тестов, фаззер находил их в 5 и в 1 сценарии из 5000. Поэтому я написал два направленных теста. Первый ведёт кластер по последовательности из рис. 8 статьи [1] и повторяет её 25 раз, потому что победитель выборов зависит от случайных таймеров. Второй заставляет узел проголосовать, упасть и вернуться, а потом просит у него голос в том же сроке второго кандидата. После этого оба варианта падают на этих тестах во всех четырёх конфигурациях.

Остальные мои ошибки сидели в обвязке. Восстановление связи узла сначала возвращало заодно и связи с изолированным лидером, из-за чего тесты на перевыборы проходили случайно. Вспомогательная функция согласия пробовала команды на отключённом старом лидере и тратила по 2 с на каждую попытку, и тест быстрого отката сначала показывал 300 с вместо 10. Первое измерение срывов в спокойном кластере показало, что PreVote ничего не меняет, потому что я считал срывом любое событие роли, а PreVote-узел возвращается в «последователи» без всяких выборов. Теперь считаются только кандидатуры и новые лидеры.

Результаты воспроизводятся побайтно. Два полных прогона всех экспериментов дали одинаковые файлы, а сборка g++ 13.3 и сборка clang 18.1 дали одинаковый вывод на проверочных запусках (по 500 и по 50 испытаний, сравнение через md5).

3.4. Окружение и как повторить

Все числа получены в облачном контейнере: Linux 6.18, Intel Xeon 2,1 ГГц, 4 ядра, g++ 13.3.0, CMake 3.28.3, Python 3.13.17 без сторонних пакетов. Время в экспериментах виртуальное и определяется только начальным значением генератора, поэтому результаты не зависят от машины. От машины зависит только скорость: 3000 испытаний холодного старта занимают 0,10–0,11 с (три запуска), 2000 фаззинг-сценариев 3,3–4,0 с. Облачный контейнер не равен ноутбуку и на ноутбуке будет иначе по скорости, но не по результатам. Все файлы в `results/` получаются командами:

```
cmake -S . -B build && cmake --build build -j4
./scripts/run_checks.sh      # тесты, фаззинг, сломанные варианты
./scripts/run_all.sh         # сырые данные экспериментов (около 100 с на 4 ядрах)
python3 -I scripts/mc_tables.py
python3 -I scripts/summary.py > results/summary.txt
python3 -I scripts/figures.py
```

4. Результаты

4.1. Формула Ongaro при $p = 0$

Сначала я проверил, что моя реализация вообще даёт то, что предсказывает [2]. При фиксированной задержке 10 мс и 20 000 испытаний на клетку я сравнил долю срабатываний, в которых первый срок не дал лидера, с формулой (1). Для падения лидера в кластере из пяти (рабочих четыре) и для холодного старта (рабочих пять) совпадение видно на рис. 1(а) и в таблице 2: из 33 сравнений наибольшее отклонение составило 2,0 стандартной ошибки. При переменной задержке 5–10 мс формулы нет, и я написал вторую реализацию на Python по описанию симуляции из [2]. Она с C++ не делится кодом и дала те же числа (рис. 1(б)). Формула (1) с максимальной задержкой 10 мс завышает долю раскола при $W \le 40$ мс и занижает её при $W \ge 50$ мс. Ongaro пишет, что граница по максимальной задержке верна примерно в 78% его точек [2], и у меня при широких диапазонах она нарушается.

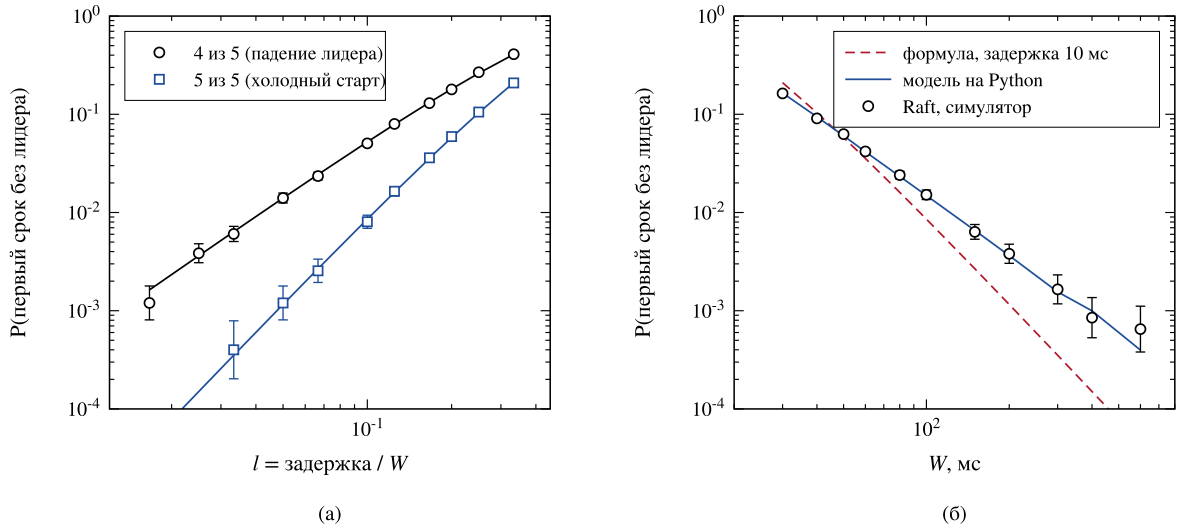


Рис. 1. Вероятность того, что первый срок не дал лидера, при $p = 0$. (а) Фиксированная задержка 10 мс; линии показывают формулу (1), маркеры показывают симулятор, усы дают 95-процентные интервалы Уилсона по 20 000 испытаниям. (б) Задержка 5–10 мс, холодный старт на пяти узлах: симулятор Raft, независимая модель на Python (400 000 испытаний на точку) и формула (1) для задержки 10 мс.

Таблица 2. Проверка формулы (1), 20 000 испытаний на клетку. $l = 10$ мс / W. В столбцах «сим.» стоит доля испытаний, где первый срок не дал лидера.

W, мс	l	падение, сим.	падение, формула	холодный, сим.	холодный, формула	5–10 мс, сим.	5–10 мс, Python
30	0,333	0,409	0,407	0,209	0,210	0,163	0,165
50	0,200	0,179	0,181	0,059	0,058	0,063	0,060
100	0,100	0,051	0,052	0,0080	0,0086	0,0152	0,0149
200	0,050	0,0140	0,0140	0,0012	0,0012	0,0038	0,0036
400	0,025	0,0039	0,0036	0,0001	0,0002	0,0008	0,0010

4.2. Сравнение с измерениями из статьи Raft

Рис. 2(а) и таблица 3 показывают простой после падения лидера, как на рис. 16 статьи [1]. Без случайности ($W = 0$) лидер не выбран ни в одном из 1000 испытаний за 60 с виртуального времени, что согласуется с «больше 10 секунд» в [1]. С 50 мс случайности худший случай из 1000 у меня 446 мс при задержке 2 мс, у Ongaro 513 мс. Количественно медиана для $W = 5$ мс у меня зависит от за-

держки. При односторонней задержке 5–10 мс она равна 1678 мс, у Ongaro 287 мс. Я подобрал задержку: при 2 мс получилось 262 мс. Подбор сделан по этому одному числу после того, как расхождение обнаружилось, поэтому как независимое подтверждение его считать нельзя. Из этого я вывожу только то, что эффективная задержка запроса голоса в измерениях [1] была ближе к 2 мс, чем к заявленным 15 мс времени рассылки (это моё предположение). С таймаутом 12–24 мс среднее у меня 19,8 мс при максимуме 74 мс, а у Ongaro 35 и 152 мс. Мой диск мгновенный, у него записи на SSD.

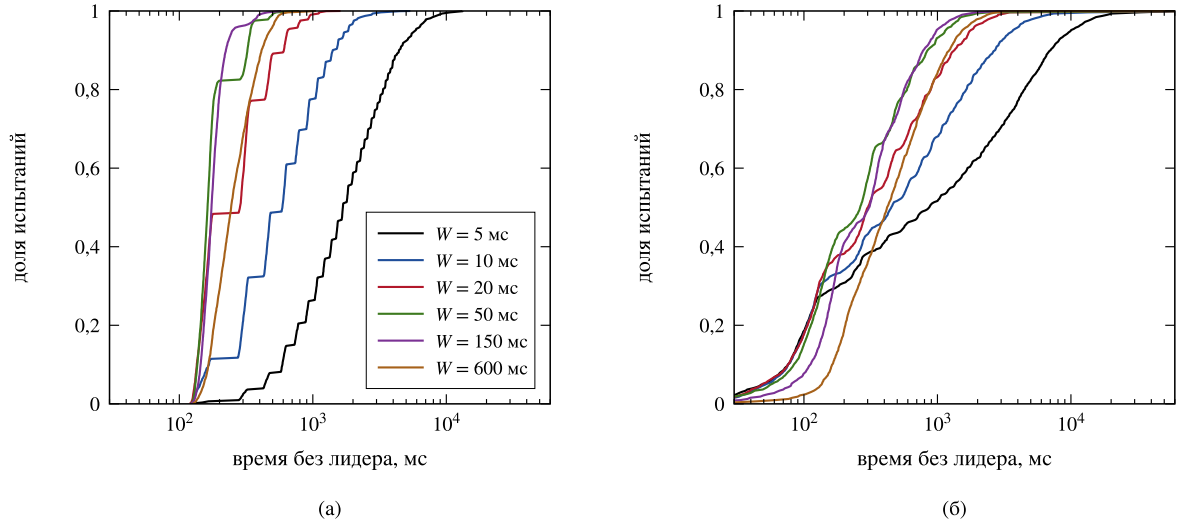


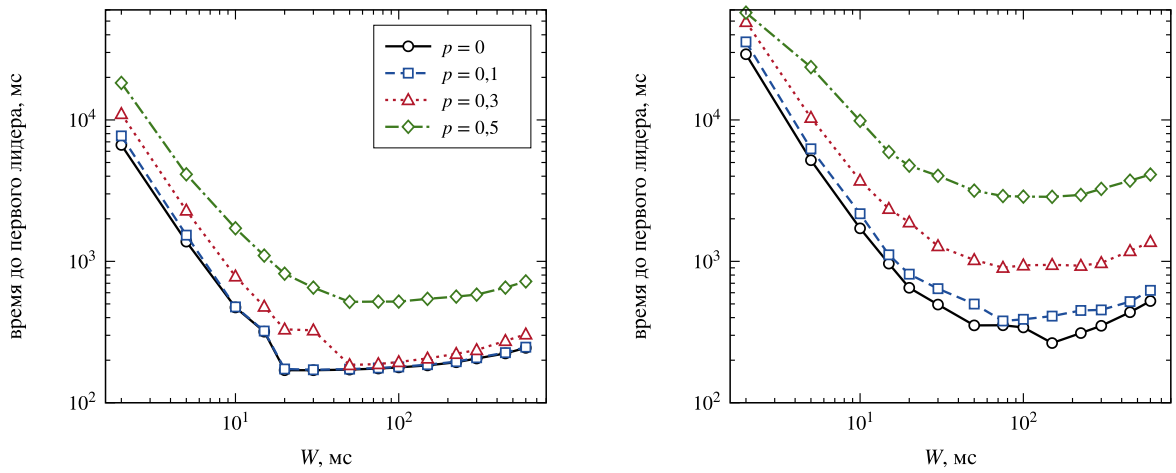
Рис. 2. Время без лидера после падения лидера, 2000 испытаний на кривую, $T = 150$ мс, задержка 5–10 мс. (а) Потерь нет, как на рис. 16 статьи [1]. (б) Потеря 30% сообщений. Обозначения в обеих панелях одинаковы.

Таблица 3. Простой после падения лидера при $p = 0$, $T = 150$ мс, 1000 испытаний, мс. Слева односторонняя задержка 2 мс, справа 5–10 мс. При $W = 0$ лидер не выбран ни в одном испытании за 60 с.

W, мс	задержка 2 мс			задержка 5–10 мс		
	медиана	99-й проц.	максимум	медиана	99-й проц.	максимум
1	3007	14708	25240			
5	262	908	1354	1678	8697	13273
20	136	309	324	284	959	1612
50	141	183	446	163	496	789
150	155	240	267	174	393	579

4.3. Время до лидера при потерях

Теперь потери. Холодный старт (все серверы стартуют с пустым журналом одновременно) даёт чистый срез первого срока. Рис. 3 показывает медиану и 99-й процентиль времени до первого лидера в зависимости от W. Все кривые имеют одну форму: при узком диапазоне время велико из-за раскола, около $W = 20$–50 мс наступает излом, дальше время растёт линейно, потому что ожидание определяется самим таймаутом. Положение излома от потерь почти не зависит. При $p = 0$ медиана достигает минимума 170 мс при $W = 20$–30 мс, при $p = 0{,}5$ минимум в 517 мс лежит около 50–100 мс. Это 3–7 односторонних задержек в соответствии с диссертацией [2]. Меняется высота плато и хвост: при $p = 0{,}5$ 99-й процентиль составляет 2,9–3,3 с вместо 0,35 с.



(а) медиана

(б) 99-й процентиль

Рис. 3. Время до первого лидера при холодном старте, 3000 испытаний на точку, $T = 150$ мс, задержка 5–10 мс. (а) Медиана. (б) 99-й процентиль.

Таблица 4. Холодный старт, $T = 150$ мс, 3000 испытаний. P_1 означает долю испытаний, где лидер выбран в первом сроке. «Устоявшийся» лидер означает, что секунду после него не было ни кандидатов, ни новых выборов; в 60 с это не наступило в ячейках, помеченных звёздочкой.

p	W, мс	P_1	медиана, мс	99-й проц., мс	устоявшийся, медиана, мс	лидеров до устоя
0	10	0,114	472	1711	472	1,0
0	50	0,941	172	353	172	1,0
0	150	0,994	184	264	184	1,0
0,1	50	0,869	173	499	174	1,1
0,1	150	0,949	186	410	187	1,0
0,1	600	0,973	247	624	248	1,0
0,3	50	0,547	185	1023	1516	4,4
0,3	150	0,649	206	945	497	1,9
0,3	600	0,689	305	1374	351	1,2
0,5	50	0,213	517	3163	*	37
0,5	150	0,268	543	2860	14910	19
0,5	600	0,299	721	4111	1716	2,2

Рис. 4 показывает главное отличие от $p = 0$. Без потерь вероятность успеха первого срока приближается к единице при росте W, потому что конкуренты расходятся во времени. С потерями кривые выходят на плато ниже единицы: при $p = 0{,}3$ около 0,69, при $p = 0{,}5$ около 0,30. Расширение диапазона за пределы 100–150 мс уже ничего не даёт, потому что срок проваливается не из-за столкновения кандидатов. Одинокий кандидат в кластере из пяти должен получить два голоса из четырёх, и каждый голос проходит туда и обратно с вероятностью $(1 - p)^2$:

$$P_{\text{одиноч}}(p) = \Sigma_{k=\lfloor n/2 \rfloor}^{s-1} \, \mathrm{C}(s{-}1, k)\, r^k (1 - r)^{s-1-k}, \quad r = (1 - p)^2. \tag{2}$$

Формула (2) даёт 0,976 при $p = 0{,}1$, 0,672 при 0,3 и 0,262 при 0,5. Плато на рис. 4 лежат близко, но не точно: при $p = 0{,}5$ и $W = 600$ мс у симулятора 0,280, у (2) 0,262, потому что второй кандидат, чьи запросы дошли, всё ещё может выиграть в том же сроке. Для точных значений я пользовался независи-

мой моделью на Python (серые линии на рис. 4). На 12 точках по 20 000 и 60 000 испытаний она отличается от симулятора не больше чем на 0,004.

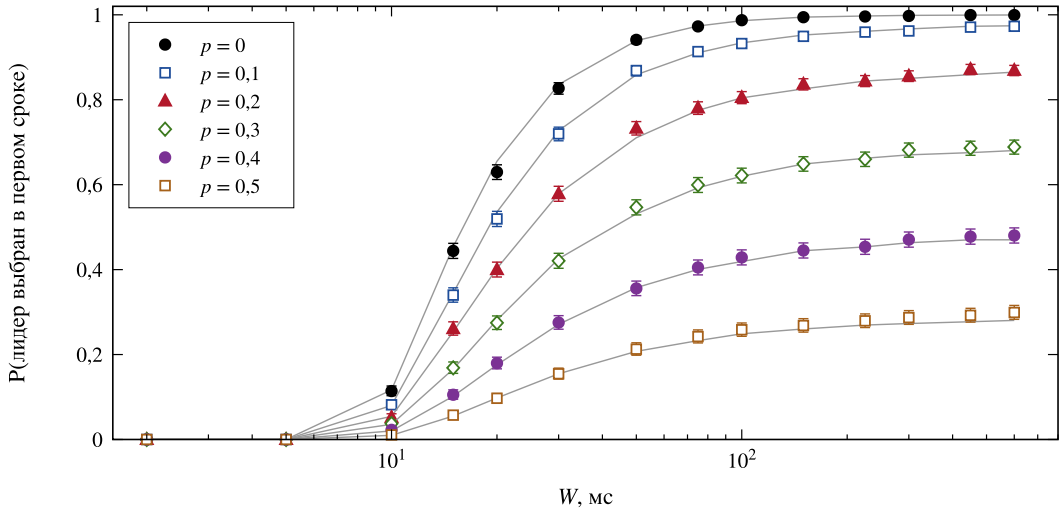


Рис. 4. Вероятность выбрать лидера в первом сроке. Маркеры: симулятор Raft с 95-процентными интервалами по 3000 испытаний. Серые линии: независимая модель на Python по 30 000 испытаний на точку.

4.4. Оценка числа сроков и память между сроками

Ongaro [2] считает число сроков геометрическим. Если q означает вероятность успеха срока, то среднее число проваленных сроков равно $(1 - q)/q$, а среднее время до лидера выражается как

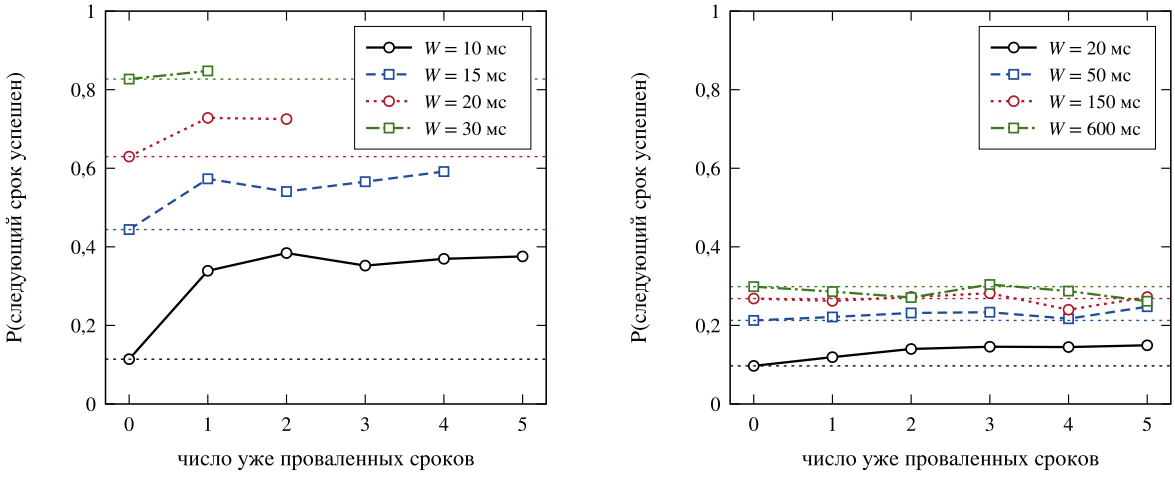
$$E[t] = a + (1/q - 1)\, b, \quad a = T + W/(s{+}1) + 2d, \quad b = T + W/(s{+}1) + d. \tag{3}$$

Здесь d означает среднюю одностороннюю задержку (7,5 мс), а q я беру равным измеренному P_1. Это упрощённая запись формулы из раздела 9.3 [2] без записи на диск. В моих данных формула (3) попадает в измеренное среднее с ошибкой до 6% при $W \le 150$ и любых p, и с ошибкой минус 5% при $p = 0{,}3$ и $W = 600$ мс и минус 14% при $p = 0{,}5$ и тех же 600 мс. Почему ошибка растёт при широком диапазоне, я не выяснял.

Геометрическое допущение выполняется при широком диапазоне, и это верно даже при $p = 0{,}5$. Вероятность успеха срока, условная на число уже проваленных, на рис. 5(б) для $W \ge 50$ мс остаётся на уровне первого срока в пределах шума (0,24–0,28 при $W = 150$ мс для всех сроков до пятого). Допущение ломается при узком диапазоне, и потери для этого не нужны (рис. 5(а)). При $p = 0$ и $W = 10$ мс первый срок успешен в 11% случаев, а все следующие примерно в 34–38%. Таблица 5 переводит это в число сроков: геометрическая оценка завышает среднее число проваленных сроков в 3,1–3,6 раза при $W = 10$ мс, в 1,1–1,3 раза при 15–20 мс и не больше чем на 3% при $W \ge 30$ мс без потерь. С потерями при $W = 20$ мс завышение остаётся 1,3–1,4 раза, а при $W \ge 50$ мс исчезает.

Таблица 5. Среднее число проваленных сроков до лидера: измерено и по геометрической оценке $(1-q)/q$ с измеренным q. Холодный старт, 20 000 испытаний для фиксированной задержки, 3000 для остальных.

условия	q	измерено	геометрически	отношение
$p = 0$, задержка 7,5 мс, $W = 10$	0,105	2,36	8,55	3,6
$p = 0$, задержка 7,5 мс, $W = 15$	0,504	0,80	0,99	1,2
$p = 0$, задержка 7,5 мс, $W = 20$	0,728	0,35	0,37	1,1
$p = 0$, задержка 5–10 мс, $W = 10$	0,114	2,47	7,77	3,1
$p = 0$, задержка 5–10 мс, $W = 30$	0,827	0,203	0,209	1,0
$p = 0{,}3$, $W = 20$	0,275	2,04	2,64	1,3
$p = 0{,}3$, $W = 150$	0,649	0,531	0,541	1,0
$p = 0{,}5$, $W = 20$	0,097	6,47	9,31	1,4
$p = 0{,}5$, $W = 150$	0,268	2,72	2,73	1,0
$p = 0{,}5$, $W = 600$	0,299	2,47	2,35	0,95



(а) потерь нет

(б) потеря половины сообщений

Рис. 5. Вероятность успеха очередного срока при условии, что до него уже провалилось указанное число сроков. Пунктир показывает значение первого срока, то есть то, что предсказывает геометрическое распределение. Точки не показаны там, где в выборке меньше 100 испытаний. (а) Потерь нет, задержка 5–10 мс. (б) Потеря половины сообщений.

Причину я объясняю так, и это моё предположение, проверенное только частично. После провала первого срока таймеры серверов стартуют не одновременно: у кандидата с момента его срабатывания, у проголосовавших с момента выдачи голоса. Разброс начальных моментов порядка W добавляется к случайному таймауту, и следующий срок оказывается фактически шире. Подтверждает это то, что эффект есть и при фиксированной задержке 7,5 мс (рис. 5 и таблица 5), то есть дело не в дрожании сети. Прямой эксперимент с принудительной синхронизацией таймеров после провала я не ставил.

4.5. Устоявшийся лидер: потери heartbeat важнее потерь запросов

Первый избранный лидер при потерях может не удержаться. Если подряд потерялись все heartbeat, которые успели бы прийти до таймаута, последователь начинает выборы и свергает лидера, у которого с остальными связь хорошая. Рис. 6 показывает время до «устоявшегося» лидера (секунда без кандидатов после выборов), и оно резко отличается от времени до первого. При $p = 0{,}3$ и $W = 150$ мс

медиана равна 497 мс против 206 мс до первого лидера, а до устоя проходит в среднем 1,9 лидера. При $p = 0{,}5$ и $T = 150$ мс кластер за минуту не устаивается в 6% случаев при $W = 150$ мс и в 85–99% случаев при $W \le 50$ мс.

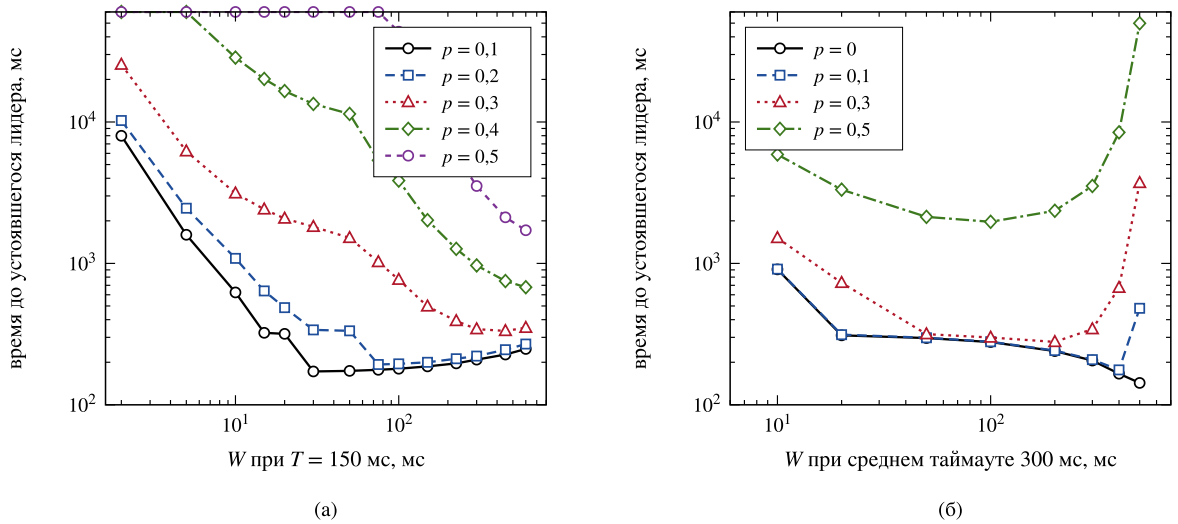


Рис. 6. Медиана времени до устоявшегося лидера, 3000 испытаний на точку, потолок 60 с. (а) $T = 150$ мс, W растёт, поэтому растёт и средний таймаут. (б) Средний таймаут зафиксирован на 300 мс ($T = 300 - W/2$), меняется только ширина диапазона.

На рис. 6(а) время до устоя падает с ростом W почти везде, а это противоположно кривым первого лидера на рис. 3. Чтобы отделить ширину от среднего таймаута, я поставил эксперимент с одинаковым средним 300 мс (рис. 6(б)). Там при $p = 0{,}3$ медиана до устоя минимальна при $W = 100$–200 мс (279–299 мс) и растёт до 3710 мс при $W = 500$ мс, когда нижняя граница T падает до 50 мс. Для коллизий кандидатов важна ширина, для стойкости к потере heartbeat важна нижняя граница T, и поэтому оптимальную ширину при потерях нельзя выбирать по одному критерию.

4.6. Спокойный кластер

Модель срывов простая. Последователь сбрасывает таймер при приёме heartbeat и тянет x из $[T, T + W]$. До момента x успевают прийти $m(x) = \lceil x/H \rceil - 1$ heartbeat, и таймер срабатывает, если все они потеряны. Вероятность срабатывания за цикл равна $q_c = E_x[p^{m(x)}]$, средняя длина цикла равна $\ell = E_x[x p^m + \Sigma_{j \le m}\, jH(1-p)p^{j-1}]$, а среднее время до первой лишней кандидатуры среди четырёх последователей оценивается как

$$E[t_{\text{срыв}}] \approx \ell \,/\, (4\, q_c). \tag{4}$$

В эксперименте кластер из пяти без потерь выбирает лидера, ждёт секунду, затем включаются потери, и фиксируется время до первой кандидатуры или нового лидера (300 испытаний, потолок 600 с). Рис. 7 и таблица 6 сравнивают измерение с (4). В 19 из 26 клеток, где срывы наблюдались хотя бы в 200 испытаниях из 300, модель попадает в измерение с ошибкой до 20% (в таблице 6 приведена часть клеток). В пяти клетках с самыми короткими временами (до 0,6 с) она занижает среднее на 21–37%. Моё предположение состоит в том, что модель не учитывает, что первое срабатывание не может случиться раньше T, но отдельной проверки у меня нет. В двух клетках при $p = 0{,}05$ модель завышает среднее на 27%, причину я не искал. Независимость потерь я задал сам, и эти оценки перенести на сети с пачками потерь нельзя.

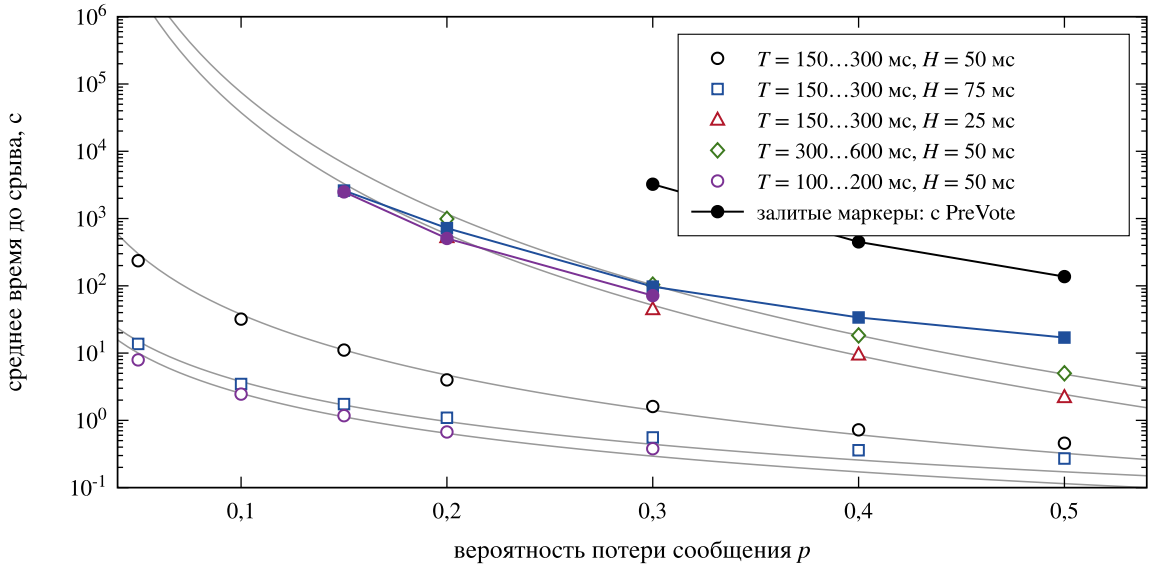


Рис. 7. Среднее время до первого срыва спокойного кластера. Маркеры: симулятор, оценка по суммарной экспозиции (время деленное на число срывов), 300 испытаний, показаны клетки не меньше чем с 20 срывами. Серые линии: формула (4). Залитые маркеры: то же с PreVote, показаны клетки не меньше чем с 10 срывами.

Таблица 6. Спокойный кластер из пяти, среднее время до первого срыва, с. «Модель» означает формулу (4). Для PreVote указано число срывов из 300 испытаний по 600 с; если срывов не было, дана нижняя граница по правилу трёх.

$T…T+W$, мс	H, мс	p	симулятор	модель	PreVote: срывов	PreVote, среднее
150…300	50	0,1	32,0	37,5	1	> 60 000
150…300	50	0,2	4,0	4,7	1	> 60 000
150…300	50	0,3	1,6	1,4	51	3237
150…300	50	0,5	0,46	0,32	300	137
150…300	25	0,3	45,0	51,5	0	> 60 000
300…600	50	0,3	104	103	0	> 60 000
300…600	50	0,5	5,0	4,9	0	> 60 000

Из этого следует практическое наблюдение. Стандартная рекомендация из [1] (150–300 мс, heartbeat вдвое-втрое чаще минимального таймаута) при потере 10% сообщений даёт срыв кластера в среднем раз в 32 секунды в моей модели, а при 20% раз в 4 секунды. Таблица 7 показывает по формуле (4), какое T при $W = T$ нужно, чтобы срыв случался в среднем реже заданного. Формулой я пользуюсь в области, где она проверена (времена от десятых долей секунды до 10^3 с), а значения за пределами проверки (сутки) являются экстраполяцией.

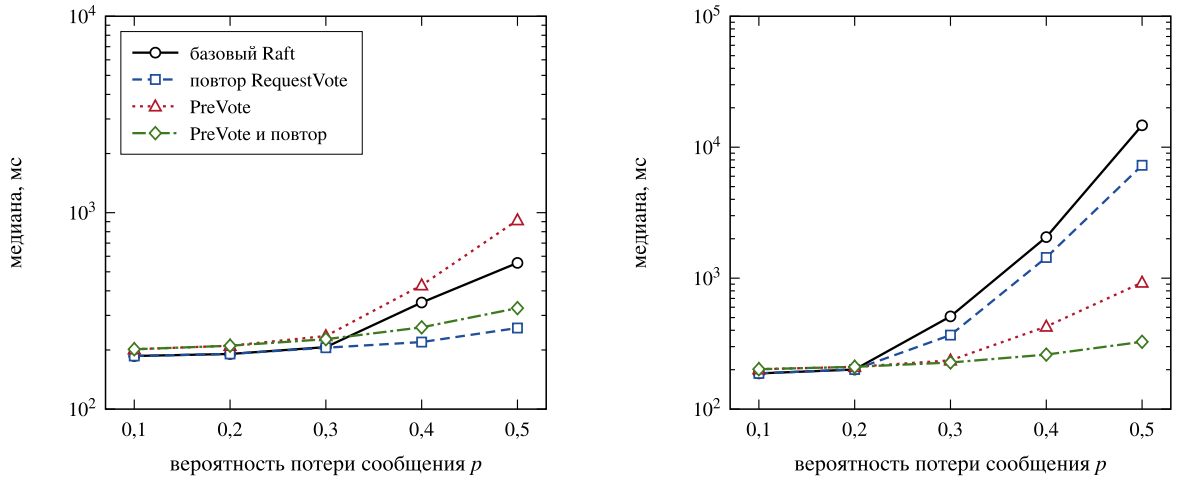
Таблица 7. Минимальное T в мс при $W = T$, при котором формула (4) даёт среднее время до срыва не меньше цели. Значения для суток лежат за областью проверки.

H, мс	цель	$p = 0{,}01$	0,05	0,1	0,2	0,3
50	1 мин	100	145	170	230	290
50	1 час	150	200	250	340	440
50	1 сутки	175	250	310	435	560
100	1 час	285	385	475	640	820

4.7. PreVote и повтор запросов

PreVote в спокойном кластере (таблица 6) увеличивает среднее время до срыва на два-три порядка. При $p = 0{,}3$ и $H = 50$ мс оно выросло с 1,6 с до 3237 с (51 срыв на 300 прогонов по 600 с), а при $p = 0{,}5$ с 0,46 с до 137 с. Причина простая: проба проходит, только если большинство серверов само не слышало лидера дольше минимального таймаута, а одиночный глухой последователь такого большинства не набирает. Рис. 8 и таблица 8 показывают цену. При $p = 0{,}5$ PreVote приносит время до устоявшегося лидера 926 мс вместо 14 689 мс, но простой после падения лидера растёт с 877 до 2459 мс по медиане, потому что проба занимает лишний раунд и тоже может потеряться.

Повтор RequestVote в пределах срока убирает именно тот провал, который не лечится расширением диапазона. При $p = 0{,}5$ и $W = 150$ мс вероятность успеха первого срока вырастает с 0,27 до 0,73, а медиана простоя после падения лидера падает с 877 до 222 мс. Повтор сам по себе не спасает от срывов heartbeat: время до устоявшегося лидера почти не меняется (7275 мс против 14 689 мс). Вместе два приёма дают лучший результат: при $p = 0{,}5$ медиана до устоявшегося лидера 327 мс, простой после падения 500 мс. Нагрузку повторов сообщениями я не измерял. Повтор у меня только для настоящих запросов голоса, а пробы PreVote не повторяются, и вполне вероятно, что часть задержки PreVote при потерях можно убрать повтором проб.



(а) до первого лидера

(б) до устоявшегося лидера

Рис. 8. Варианты алгоритма при $W = 150$ мс, $T = 150$ мс, холодный старт, 2000 испытаний на точку. (а) Медиана времени до первого лидера. (б) Медиана времени до устоявшегося лидера.

Таблица 8. Варианты при $W = 150$ мс, медиана, мс. «Простой» означает время без лидера после падения лидера, «устоявшийся» означает холодный старт до устоявшегося лидера (потолок 60 с).

p	вариант	простой	99-й проц.	устоявшийся	лидеров до устоя
0,3	базовый	308	1529	511	1,9
0,3	повтор	177	583	367	2,0
0,3	PreVote	401	2049	236	1,0
0,3	PreVote и повтор	248	607	227	1,0
0,5	базовый	877	6359	14689	18,9
0,5	повтор	222	1093	7275	23,7
0,5	PreVote	2459	15410	926	1,0
0,5	PreVote и повтор	500	2117	327	1,0

Размер кластера и масштаб задержки сдвигают всё в ожидаемую сторону, и я привожу только то, что видно в данных. Положение излома определяется отношением W к задержке: при задержке 0,5–1,5 мс P_1 достигает 0,88 уже при $W = 5$ мс, а при 20–40 мс доходит лишь до 0,89 при $W = 150$ мс. С ростом кластера при $p = 0{,}3$ и $W = 150$ мс P_1 падает с 0,76 (три узла) до 0,65 (пять), 0,58 (семь) и 0,54 (девять), потому что нужно больше голосов, и каждый может потеряться.

5. Где это может быть неверно

Потери в моей модели независимы между сообщениями и между связями, а в настоящих сетях они идут пачками. Для формулы (4) это существенно: при пачках подряд потерянные heartbeat встречаются чаще, и срывы случатся раньше. Задержка равномерная и не зависит от нагрузки, часы идеальны, диск мгновенный, процессор не ограничивает обработку. Эти упрощения не те, что в [1], где измерения шли на реальных серверах с записью на диск, и отчасти поэтому мои абсолютные времена нельзя сравнивать с их цифрами напрямую.

Реализация проверена на корректность тестами и фаззером, но не формально. Фаззер слабо ловит редкие ошибки (одно из двух самых тонких нарушений он нашёл в 0,1% сценариев), поэтому «ноль нарушений на 40 000 сценариев» говорит о надёжности меньше, чем кажется. Направленные тесты написаны под известные мне ошибки и других не поймают. Реализация PreVote сделана по описанию из [2], я не сверял её с промышленными реализациями. Повтор запросов придуман мной и не изучался с точки зрения нагрузки.

Объяснение памяти между сроками (разброс стартовых моментов таймеров) подтверждено лишь тем, что эффект переживает фиксированную задержку, а прямой проверки у меня нет. Объяснение расхождения медианы 287 мс в [1] эффективной задержкой 2 мс опирается на подбор параметра по одному числу. Сравнение с [1] вообще неточное, потому что в [1] использовались серверы с разной длиной журнала, чтобы часть кандидатов была неподходящей, а у меня все журналы пустые при холодном старте.

Большая часть данных получена на пяти узлах. Для 3, 7 и 9 узлов я проверил только P_1 и медиану при двух значениях W. Время всех экспериментов виртуальное, поэтому цифры о скорости (0,1 с на 3000 испытаний, 3,3–4,0 с на 2000 фаззинг-сценариев) относятся к одной облачной машине.

6. Открытые вопросы

Не проверены потери пачками (например, модель Гилберта–Эллиотта) и асимметричные потери по связям. Для обоих случаев формулу (4) стоило бы пересчитать. Остаётся проверить повтор проб PreVote, динамический выбор таймаута в зависимости от измеренной доли потерь (в [2] такая идея на-

звана открытой) и дрейф часов. Мой вывод о том, что нижняя граница T определяет стойкость к потере heartbeat, а ширина W определяет коллизии, стоило бы превратить в совместную оптимизацию пары (T, W) под заданные p и задержку. Я этого не делал и ограничился двумя срезами (рис. 6).

Литература

[1] Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). Stanford University, 2014; сокращённая версия: USENIX ATC 2014. Читал расширенную версию, raft.github.io/raft.pdf (копия в репозитории raft/raft.github.io).

[2] Ongaro D. Consensus: Bridging Theory and Practice. PhD dissertation, Stanford University, 2014. Глава 9 «Leader election evaluation», разделы 9.1–9.6. Лицензия CC BY, github.com/ongardie/dissertation.

[3] Fischer M. J., Lynch N. A., Paterson M. S. Impossibility of distributed consensus with one faulty process // Journal of the ACM 32(2), 1985, p. 374–382. Ссылка проверена только по библиографии [2], саму работу я не открывал.

[4] Howard H. ARC: Analysis of Raft Consensus. Technical Report UCAM-CL-TR-857, University of Cambridge Computer Laboratory, July 2014. Знаю только по ссылке в [2].

[5] MIT 6.5840 Distributed Systems, лабораторная по Raft, pdos.csail.mit.edu/6.824/labs/. Страницу открыть не удалось, содержание знаю по памяти и по краткому пересказу в поисковой выдаче.